Lab 11: Government Data APIs

INFO 4617/5617 · Week 11 · Chapter 11, Government Data APIs

This week every class is live coding: one lab in three parts, one a day. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll use four public APIs, and each one shows an idea that holds for APIs in general:

- **Part 1, Monday:** the **Census Bureau**, and what can I ask for?;
- **Part 2, Wednesday: FRED**, the Federal Reserve Bank of St. Louis's data service, and let the server compute; then the **Energy Information Administration (EIA)**, and let the server filter;
- **Part 3, Friday: Congress.gov**, and follow the links; then a join of two of the APIs.

Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Request four free keys and set them as environment variables before you start Jupyter (Missing Manual Chapter 34 shows how):

- `CENSUS_KEY`, from the Census Bureau, for Monday; click the activation link in its email before you use it;
- `FRED_KEY`, from FRED, and `EIA_KEY`, from the EIA, for Wednesday;
- `DATAGOV_KEY`, from api.data.gov, for Friday's Congress.gov.

Each part reads only the keys it needs.

On Wednesday and Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**), or start at that day's setup step.

> **Instructor key.** Part 1 was run live on 2026-10-25 with `labs/tools/run_key.py`, and every one of its cells ran. Its figures come from the 2018–2022 American Community Survey's five-year estimates, and a rerun gives the same numbers; Step 2's list of geographies could change only with a new dataset. Step 3's request failed twice at the course machine's proxy and went through on the third try. Parts 2 (FRED and the EIA) and 3 (Congress.gov) aren't written yet: each is added, and run, before its day. The lab follows the textbook at commit `d9306b7`, before this week's revision of chapter 11; where it departs from the chapter, the step says so. The chapter still names the key `CENSUS_API_KEY`; the course uses `CENSUS_KEY`.

Part 1 — Monday: the Census, and what can I ask for?

Every API answers only the questions it was built for, in its own words. Before we can ask the Census Bureau anything, we have to find out what it can tell us (its variables) and about what (its geographies). The Census API answers both questions itself, in catalogs we can request like any other data. Finding out what an API offers, from the API, is called discovery.

Step 0 — Set up, and keep the key out of the notebook (5 minutes)

An API key identifies you, counts your requests, and is yours alone. A key typed into a notebook travels with the notebook: into a GitHub repository, a class submission, a classmate's laptop. So read the key from an environment variable, as the chapter's "API Key Management" section does. The chapter checks each key and raises its own error; `os.environ["NAME"]` does the same in one line, since a missing key raises a `KeyError` that names it. (The chapter also shows a second way, a `.gitignore`d `api_keys.json` file. It names the variable `CENSUS_API_KEY`; this course uses `CENSUS_KEY`, and today we

need only that one.) Define the User-Agent too: these APIs ask for a key, but every request still says who sent it, in the book's form.

```
# Import os, requests, pandas (as pd), and matplotlib.pyplot (as plt)
# Define HEADERS, whose "User-Agent" is "WebDataScience/1.0 (your-email@colorado.edu)", with your email
# Read CENSUS_KEY into census_key with os.environ[...]; never print the key
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
census_key = os.environ["CENSUS_KEY"]
print("Census key found")
```

```
Census key found
```

💬 Discuss

- In week 10 we told Wikipedia who we are with a User-Agent. The Census Bureau also asks for a key: what can a key do that a User-Agent can't?
- Where could a key typed into this notebook end up: a commit, a screenshot, a shared notebook? Which of those could you take back?
- If `CENSUS_KEY` isn't set, what does this cell do? Why is that better than finding out at the first request?

Step 1 — What can I ask about? The variable catalog (7 minutes)

The American Community Survey's thousands of variables come in groups (the chapter's "Navigating the Variable Catalog"). Group `B25034` is "Year Structure Built", and the group's catalog is itself an API: send it a GET with `requests.get()` and our `HEADERS`, and read the answer with `.json()`. Print only the estimates, the codes ending in `E`: the chapter skips the margins of error (`M`), but the group also holds annotation variables (`EA`, `MA`) that would triple the list. Browse all the groups at api.census.gov/data/2022/acs/acs5/groups.html.

```
# Request "https://api.census.gov/data/2022/acs/acs5/groups/B25034.json" with HEADERS; read the JSON into variables
# Loop over sorted(variables["variables"].items()), and print each code that ends in "E" with its label
variables_url = "https://api.census.gov/data/2022/acs/acs5/groups/B25034.json"
variables = requests.get(variables_url, headers=HEADERS).json()

for var_id, var_info in sorted(variables["variables"].items()):
    if var_id.endswith("E"):
        print(f"{var_id}: {var_info['label']}")
```

```
B25034_001E: Estimate!!Total:
B25034_002E: Estimate!!Total:!!Built 2020 or later
B25034_003E: Estimate!!Total:!!Built 2010 to 2019
B25034_004E: Estimate!!Total:!!Built 2000 to 2009
B25034_005E: Estimate!!Total:!!Built 1990 to 1999
B25034_006E: Estimate!!Total:!!Built 1980 to 1989
B25034_007E: Estimate!!Total:!!Built 1970 to 1979
B25034_008E: Estimate!!Total:!!Built 1960 to 1969
B25034_009E: Estimate!!Total:!!Built 1950 to 1959
B25034_010E: Estimate!!Total:!!Built 1940 to 1949
B25034_011E: Estimate!!Total:!!Built 1939 or earlier
```

```
NAME: Geographic Area Name
```

💬 Discuss

- What do the parts of `B25034_002E` tell you: the table, the line, and the final letter? Why would an estimate need an annotation variable beside it?
- In week 6, The Numbers hid its table among decoys. How is picking a variable from this catalog like that, and how is it different?
- Every cell today assumes its request works. If the Census Bureau answered a 429, or a web page with status 200, which line would fail, and would week 10's `raise_for_status()` have caught either?

Step 2 — Where can I ask about? The geography catalog (6 minutes)

The variables are half of a question; the other half is a place. The Census divides the country into nested geographies (states, counties, places, tracts, and more), and the dataset's geography catalog lists every level it can answer for, and what each one requires: the larger geography you must name with it. It is an API too, at `geography.json`. Print the first levels' names and requirements. The chapter's "Querying for Specific Places" uses one of them, `place`: find it, and what it needs.

```
# Request "https://api.census.gov/data/2022/acs/acs5/geography.json" with HEADERS; read the JSON
↪  into geographies
# Loop over the first 13 entries of geographies["fips"], and print each one's "name" and its
↪  .get("requires")
geography_url = "https://api.census.gov/data/2022/acs/acs5/geography.json"
geographies = requests.get(geography_url, headers=HEADERS).json()

for level in geographies["fips"][:13]:
    print(f"{level['name']}: requires {level.get('requires')}")
```

```
us: requires None
region: requires None
division: requires None
state: requires None
county: requires ['state']
county subdivision: requires ['state', 'county']
subminor civil division: requires ['state', 'county', 'county subdivision']
place/remainder (or part): requires ['state', 'county', 'county subdivision']
tract: requires ['state', 'county']
block group: requires ['state', 'county', 'tract']
county (or part): requires ['state', 'place']
place: requires ['state']
consolidated city: requires ['state']
```

💬 Discuss

- A city is a `place`. What else must a request about a place name, and why would a `tract` need a county as well as a state?
- Steps 1 and 2 asked the API what it knows before asking it anything. Without the two catalogs, how would you have found `B25034` or `place`?
- Which catalog would you check again next year, when the Census publishes the 2023 estimates, and why might its answer change?

Step 3 — Ask about one place (5 minutes)

Step 2 says a place needs its state. Places are named by FIPS codes: Colorado is state `08`, and Boulder is place `07850`. The chapter's "Querying for Specific Places" asks for the place's name and the first few `B25034` estimates, with `get`, `for`, and `in` parameters, and the key. The answer is a list of

lists whose first row is the header, and every value is a string, so make a DataFrame and turn the numbers into numbers with `pd.to_numeric`. Keep the chapter's names, `census_url` and `numeric_cols`: we use them again.

```
# census_url = "https://api.census.gov/data/2022/acs/acs5"
# params: get="NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,B25034_005E",
 ↪  for="place:07850", in="state:08", key=census_key
# Request census_url with params and HEADERS; make df from rows 1 onward, with row 0 as the
 ↪  columns
# numeric_cols: the columns that start with "B25034"; convert them with pd.to_numeric, and show
 ↪  df
census_url = "https://api.census.gov/data/2022/acs/acs5"
params = {"get": "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,B25034_005E",
          "for": "place:07850", "in": "state:08", "key": census_key}
data = requests.get(census_url, params=params, headers=HEADERS).json()

df = pd.DataFrame(data[1:], columns=data[0])
numeric_cols = [c for c in df.columns if c.startswith("B25034")]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric)
df
```

```
                    NAME  B25034_001E  B25034_002E  B25034_003E  B25034_004E  \
0  Boulder city, Colorado        46551           68         3784         3847

   B25034_005E state  place
0         5931    08  07850
```

💬 Discuss

- In week 5, `urlparse()` took apart a Census URL just like this request's. Which parameters name the variables, which the place, and which must never appear in a printed error?
- In week 7, the Wayback Machine's CDX API sent the same shape: a list of lists with a header row. Why might an API send data this way rather than as nested dictionaries?
- Why do the numbers arrive as strings, and why must a FIPS code like `08` stay one?

Step 4 — Compare two places (12 minutes)

Several places can go in one request: `place:07850,45970` is Boulder and Longmont (the chapter's "Comparing Multiple Locations"). Change one parameter and ask again, then plot the two cities' housing by decade built, side by side. The chapter's chart includes the total among the decades, which dwarfs them; leave it out, and label the decades.

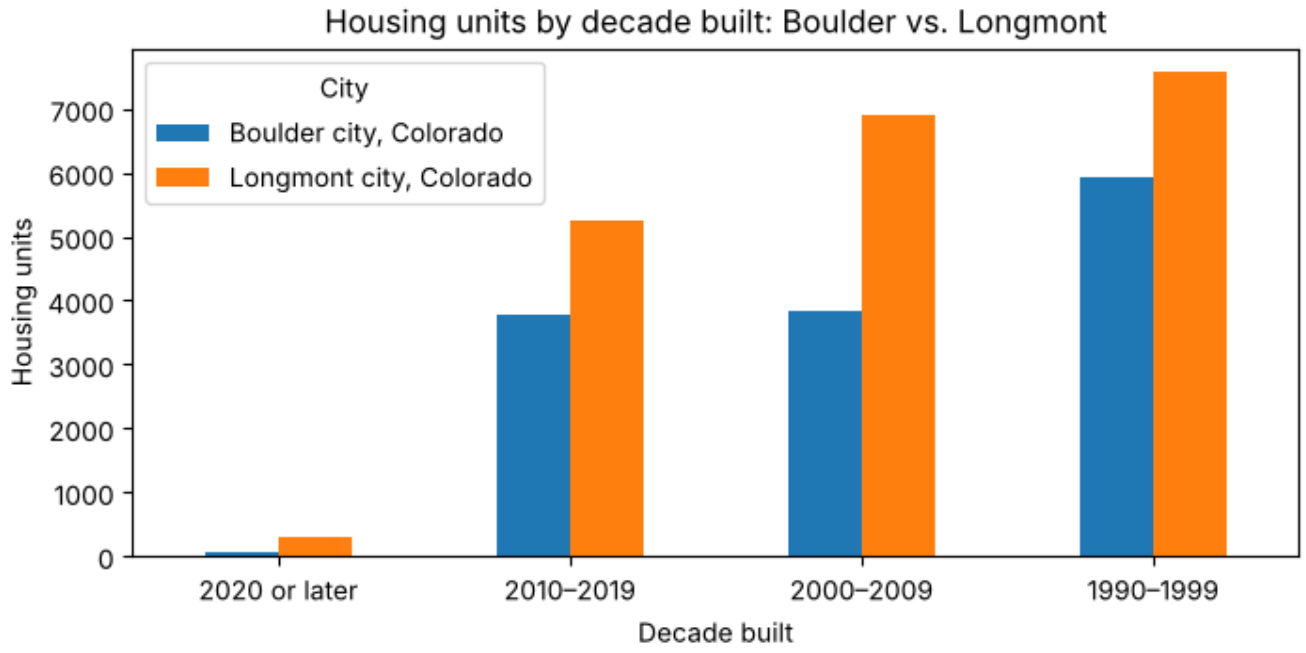
```
# Set params["for"] to "place:07850,45970" and request again
# Build df_compare as in Step 3, convert numeric_cols, and show it
params["for"] = "place:07850,45970"
data = requests.get(census_url, params=params, headers=HEADERS).json()
df_compare = pd.DataFrame(data[1:], columns=data[0])
df_compare[numeric_cols] = df_compare[numeric_cols].apply(pd.to_numeric)
df_compare
```

```
                     NAME  B25034_001E  B25034_002E  B25034_003E  \
0   Boulder city, Colorado        46551           68         3784
1  Longmont city, Colorado        41429          303         5250

   B25034_004E  B25034_005E state  place
0         3847         5931    08  07850
1         6912         7569    08  45970
```

```
# df_plot: df_compare indexed by NAME, the decade columns only (numeric_cols[1:]), transposed;
 ↪  name its rows by decade
# On plt.subplots(), plot df_plot as grouped bars, with a title, axis labels, and a legend titled
 ↪  "City"
df_plot = df_compare.set_index("NAME")[numeric_cols[1:]].T
df_plot.index = ["2020 or later", "2010–2019", "2000–2009", "1990–1999"]

fig, ax = plt.subplots(figsize=(8, 3.5))
df_plot.plot(kind="bar", ax=ax, rot=0)
ax.set(title="Housing units by decade built: Boulder vs. Longmont",
       xlabel="Decade built", ylabel="Housing units")
ax.legend(title="City")
plt.show()
```



⏱️ Microsprint (6 minutes, alone or in pairs)

A short version of Additional Exercise 1. Cities differ in size, so raw counts don't compare fairly; shares do.

Do: ask for the total (`B25034_001E`) and the three decades since 2000 (`_002E` to `_004E`) for five Colorado places, and compute the share of each city's housing built in 2000 or later. The key uses Boulder, Longmont, Fort Collins (`27425`), Lakewood (`43000`), and Castle Rock (`12415`); look up others in the FIPS code lists. **Share:** the city with the largest share, the one with the smallest, and the two shares.

```
# Microsprint: set params["get"] to NAME and B25034_001E to B25034_004E, params["for"] to five
 ↪  places, and request
# df_five: indexed by NAME, the four counts converted with pd.to_numeric
# since_2000: 002E + 003E + 004E; share: since_2000 / 001E; show share, largest first
params["get"] = "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E"
params["for"] = "place:07850,45970,27425,43000,12415"
data = requests.get(census_url, params=params, headers=HEADERS).json()
df_five = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
df_five = df_five[["B25034_001E", "B25034_002E", "B25034_003E",
 ↪  "B25034_004E"]].apply(pd.to_numeric)
```

```
since_2000 = df_five["B25034_002E"] + df_five["B25034_003E"] + df_five["B25034_004E"]
share = since_2000 / df_five["B25034_001E"]
share.sort_values(ascending=False).round(3)
```

```
NAME
Castle Rock town, Colorado     0.685
Fort Collins city, Colorado    0.341
Longmont city, Colorado        0.301
Boulder city, Colorado         0.165
Lakewood city, Colorado        0.150
dtype: float64
```

💬 Discuss

- Which of your five cities built the most of its housing since 2000, and which the least? What might explain the difference?
- Longmont out-built Boulder in every decade the chart shows. Is that a fair comparison, and what does the share add?
- These are 2018–2022 five-year estimates. What does that mean for a count labeled ”2020 or later”?

Step 5 — Sanity-check a number (8 minutes)

A variable with the right-sounding label can still be the wrong one. The chapter's ”Finding the Right Variable” ends with a practical tip: before you trust a pipeline, retrieve one value you can judge. Ask for Boulder's median household income, `B19013_001E`. Does it look like a city's median income: tens of thousands of dollars, not 7 or 70 million?

```
# params: get="NAME,B19013_001E", for="place:07850", in="state:08", key=census_key; request it
# Print the city and its median household income, with a dollar sign and commas
params = {"get": "NAME,B19013_001E", "for": "place:07850", "in": "state:08", "key": census_key}
data = requests.get(census_url, params=params, headers=HEADERS).json()
income = int(data[1][1])
print(f"{data[1][0]}: median household income ${income:,}")
```

```
Boulder city, Colorado: median household income $80,243
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Check the step's result another way. Every estimate has a margin of error, the variable with the same code ending in `M`; the American Community Survey gives it at 90% confidence.

Do: ask for `B19013_001E` and `B19013_001M` for Boulder and Longmont, and compute each city's interval: the estimate minus and plus its margin. **Share:** the two intervals, and whether they overlap.

```
# Microsprint: set params["get"] to NAME, B19013_001E, and B19013_001M, params["for"] to
 ↪  "place:07850,45970", and request
# incomes: indexed by NAME, the two columns converted; add low (estimate - margin) and high
 ↪  (estimate + margin)
# Show incomes: do the two intervals overlap?
params["get"] = "NAME,B19013_001E,B19013_001M"
params["for"] = "place:07850,45970"
data = requests.get(census_url, params=params, headers=HEADERS).json()
incomes = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
incomes = incomes[["B19013_001E", "B19013_001M"]].apply(pd.to_numeric)
incomes["low"] = incomes["B19013_001E"] - incomes["B19013_001M"]
incomes["high"] = incomes["B19013_001E"] + incomes["B19013_001M"]
incomes
```

```
                        B19013_001E  B19013_001M    low   high
NAME
Boulder city, Colorado        80243         3561  76682  83804
Longmont city, Colorado       89720         3791  85929  93511
```

💬 Discuss

- Does Boulder's median income look right? What did you compare it with?
- Did the two intervals overlap? If they had, what could you no longer say about which city's households earn more?
- Which checks like this one could a pipeline run on its own, every time it runs?

Step 6 — When there's no estimate (6 minutes)

Seventy-two of Colorado's 482 places have no estimate of median household income in the 2018–2022 survey: too few households answered for the Census Bureau to compute one. The API doesn't leave the value out. It sends a number that can't be an income, `-666666666`, and `-222222222` for its margin of error; the Bureau's notes on estimate and annotation values list what each code means. Ask for the population (`B01003_001E`), the median income, and its margin for Boulder and three small towns: Bonanza (`07571`), Campo (`11645`), and Carbonate (`12030`). Look at what comes back, then set every negative value aside with `mask()`, which turns it into a missing value, `NaN`.

```
# params: get="NAME,B01003_001E,B19013_001E,B19013_001M", for="place:07850,07571,11645,12030",
 ↪  in="state:08", key=census_key
# Request it; towns: indexed by NAME, the three columns converted with pd.to_numeric; show towns
params = {"get": "NAME,B01003_001E,B19013_001E,B19013_001M",
          "for": "place:07850,07571,11645,12030", "in": "state:08", "key": census_key}
data = requests.get(census_url, params=params, headers=HEADERS).json()
towns = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
towns = towns[["B01003_001E", "B19013_001E", "B19013_001M"]].apply(pd.to_numeric)
towns
```

```
                        B01003_001E  B19013_001E  B19013_001M
NAME
Bonanza town, Colorado           11   -666666666   -222222222
Boulder city, Colorado       106598        80243         3561
Campo town, Colorado            103   -666666666   -222222222
Carbonate town, Colorado          0   -666666666   -222222222
```

```
# Print the mean of the median-income column as it is; then mask every value below zero, and
 ↪  print the mean again
print(f"Mean with the codes in: {towns['B19013_001E'].mean():,.0f}")
towns = towns.mask(towns < 0)
print(f"Mean with them masked: {towns['B19013_001E'].mean():,.0f}")
towns
```

```
Mean with the codes in: -499,979,939
Mean with them masked: 80,243
```

```
                        B01003_001E  B19013_001E  B19013_001M
NAME
Bonanza town, Colorado           11          NaN          NaN
Boulder city, Colorado       106598      80243.0       3561.0
Campo town, Colorado            103          NaN          NaN
Carbonate town, Colorado          0          NaN          NaN
```

💬 Discuss

- What would a chart of median income have shown with `-666666666` left in? Would you have noticed?
- Why might an API send a code like this rather than leave the value empty? What does a missing value lose that the code keeps?
- Carbonate has no residents, yet it has a row. What does the Census count as a place, and what does that mean for a table of ”every place”?

End of Part 1

On Wednesday, Part 2 asks FRED to compute for us, then the EIA to filter for us; on Friday, Part 3 follows Congress.gov's links, and joins two of the week's APIs. Each part is added to this notebook before its day. Download the new copy, and start at its setup step: it imports and reads the keys again, so it doesn't need Monday's cells to have run.